



Pashov Audit Group

Polymarket V2 Security Review

April 14th 2026 - May 3rd 2026



Contents

1. About Pashov Audit Group	3
2. Disclaimer	3
3. Risk Classification	3
4. Methodology	4
5. About Polymarket V2	4
6. Executive Summary	4
7. Findings	6
Medium findings	9
[M-01] Surplus collateral can be stuck in exchange	9
[M-02] <code>OptimisticOracleModule</code> re-request reward is always zero	11
[M-03] Pre-resolved neg-risk subconditions can brick outstanding OOv2 settlements	11
[M-04] Neg-risk migration accepts legacy events with insufficient conditions for safety	13
[M-05] Batch sell matching may yield incorrect taker proceeds from <code>ctx.takerFillAmount</code>	14
Low findings	17
[L-01] Preapproved orders skip ERC-1271 revalidation at fill time	17
[L-02] Atomic NegRisk results are not constrained to single-winner shape	17
[L-03] OracleAggregator accepts undocumented partial-window NegRisk requests	18
[L-04] Migration snapshot can be diluted by post-prepare legacy questions	20
[L-05] Bridge events log router as sender	20
[L-06] Resolve <code>resolveCondition()</code> emits misleading oracle address in <code>ResultReported</code>	21
[L-07] <code>OptimisticOracleModule.currentRequestId</code> not cleared after resolution	21
[L-08] Fee-on-transfer tokens silently underfund oracle reward	22
[L-09] Admins can revoke peer admins	22
[L-10] Missing sanitation check	23
[L-11] Module replacement can fragment shared-vault liquidity for migrated position	23
[L-12] Event-level resolution pause is dead storage for NegRisk condition	25
[L-13] Admin override via <code>resolveResult</code> leaves arbitrator module stuck in <code>isActive</code>	26
[L-14] <code>OracleAggregator.disputeResult</code> is blocked by pause during active dispute window	26
[L-15] <code>InitializableRoles.addAdmin</code> contradicts the documented role management spec	27
[L-16] Per-fill <code>maxFee</code> check allows excessive fees due to lack of cumulative tracking	28
[L-17] <code>maxFeeRate</code> is set but not enforced during order matching validation	29
[L-18] Solidity rounding allows zero taking amounts	29
[L-19] Sequential nonces can block valid ramp approval	32
[L-20] Collateral token fails to reconcile decimal mismatches for assets and chains	33
[L-21] Bridged NegRisk positions may be stranded if <code>prepareConditionFromBridge</code> is first	34
[L-22] <code>ChainlinkReporterModule.report</code> allows voteCount inflation due to lack of deduplication	35
[L-23] Invalid outcomes return unusable legacy position IDs	36
[L-24] Aggregator can mark a conflicting result resolved after the target was already resolved	37
[L-25] Auto-fallback bypasses pause on the synthetic Other condition	37
[L-26] Wrong resolver setting	38
[L-27] <code>PermissionedRamp</code> emits no event on witness-signed, nonce-consuming wrap/unwrap	39
[L-28] <code>ResultReported.resolver</code> is inconsistent for contract-derived resolutions	40
[L-29] ERC-1271 signature validation forwards all available gas	41
[L-30] Compress Function Silently Destroys Dust Positions in YES Branch	42



1. About Pashov Audit Group

Pashov Audit Group consists of 40+ freelance security researchers, who are well proven in the space - most have earned over \$100k in public contest rewards, are multi-time champions or have truly excelled in audits with us. We only work with proven and motivated talent.

With over 300 security audits completed — uncovering and helping patch thousands of vulnerabilities — the group strives to create the absolute very best audit journey possible. While 100% security is never possible to guarantee, we do guarantee you our team's best efforts for your project.

Check out our previous work [here](#) or reach out on Twitter [@pashovkrum](#).

2. Disclaimer

A smart contract security review can never verify the complete absence of vulnerabilities. This is a time, resource and expertise bound effort where we try to find as many vulnerabilities as possible. We can not guarantee 100% security after the review or even if the review will find any problems with your smart contracts. Subsequent security reviews, bug bounty programs and on-chain monitoring are strongly recommended.

3. Risk Classification

Severity	Impact: High	Impact: Medium	Impact: Low
Likelihood: High	Critical	High	Medium
Likelihood: Medium	High	Medium	Low
Likelihood: Low	Medium	Low	Low

Impact

- **High** - leads to a significant material loss of assets in the protocol or significantly harms a group of users
- **Medium** - leads to a moderate material loss of assets in the protocol or moderately harms a group of users
- **Low** - leads to a minor material loss of assets in the protocol or harms a small group of users

Likelihood

- **High** - attack path is possible with reasonable assumptions that mimic on-chain conditions, and the cost of the attack is relatively low compared to the amount of funds that can be stolen or lost
- **Medium** - only a conditionally incentivized attack vector, but still relatively likely
- **Low** - has too many or too unlikely assumptions or requires a significant stake by the attacker with little or no incentive



4. Methodology

Pashov Audit Group conducts each engagement as a team that works through the codebase end-to-end: we map the system, hunt for bugs, and then collaborate with the client to remediate the findings. This report tracks every commit reviewed, every finding raised, and the resolution status of each issue. Our auditors draw on experience from private engagements and public contests, and pick their own toolset per project — static analysis, fuzzers, formal verification, and AI-assisted review — based on what fits the codebase.

5. About Polymarket V2

Polymarket V2 is a next-generation prediction market smart contract system that manages position tokens (ERC1155), collateral wrapping/unwrapping, market resolution via modular oracles, order matching, and cross-chain bridging. Positions are represented as structured IDs that encode module routing, condition identity, and outcome index directly in the token ID, eliminating storage lookups and enabling pure derivation of all relationships.

6. Executive Summary

A time-boxed security review of the **Polymarket/polymarket-v2** repository was done by Pashov Audit Group, during which **0x133**, **ChainDefenders**, **danvinci**, **montecristo**, **Shaka** engaged to review **Polymarket V2**. A total of **35** issues were uncovered.

Protocol Summary

Project Name	Polymarket V2
Protocol Type	prediction market protocol
Timeline	April 14th 2026 - May 3rd 2026

Review commit hash:

- [75f19783f8648ef8a3f5e2344f917fce3e703559](#)
(Polymarket/polymarket-v2)

Fixes review commit hash:

- [c3be835d5d6febad4e52e43739835a2adc071fc8](#)
(Polymarket/polymarket-v2)

Scope

`ERC1155TokenReceiver.sol` `InitializableRoles.sol` `Roles.sol` `BridgeBase.sol`
`IBridge.sol` `IBridgeCallback.sol` `Exchange.sol` `OrderStructs.sol`
`BridgePayloads.sol` `CrossChainTypes.sol` `ModuleIds.sol` `PositionIdLib.sol`
`BaseModule.sol` `ModuleErrors.sol` `OracleModule.sol` `BinaryModule.sol`
`IModuleCallback.sol` `INegRiskModuleCallbacks.sol` `BaseMigrationMixin.sol`



BinaryMigrationMixin.sol NegRiskMigrationMixin.sol NegRiskModule.sol
OracleAggregatorErrors.sol OracleAggregatorEvents.sol OracleModuleBase.sol
IArbitratorModule.sol IBinaryReporter.sol IDisputerModule.sol
IOptimisticOracleV2.sol IOptimisticRequester.sol IOracleAggregator.sol
IReporterModule.sol OptimisticOraclePayoutLib.sol Auth.sol Pausable.sol
OptimisticOracleModule.sol ChainlinkReporterModule.sol OracleAggregator.sol
IPositionManagerModule.sol PositionManager.sol BridgeRouter.sol
CtfRouter.sol Router.sol AutoRedeemer.sol CtfCollateralAdapter.sol
NegRiskCtfCollateralAdapter.sol CollateralOfframp.sol CollateralOnramp.sol
CollateralToken.sol PermissionedRamp.sol



7. Findings

Findings count

Severity	Amount
Medium	5
Low	30
Total findings	35

Summary of findings

ID	Title	Severity	Status
[M-01]	Surplus collateral can be stuck in exchange	Medium	Resolved
[M-02]	<code>OptimisticOracleModule</code> re-request reward is always zero	Medium	Resolved
[M-03]	Pre-resolved neg-risk subconditions can brick outstanding OOV2 settlements	Medium	Resolved
[M-04]	Neg-risk migration accepts legacy events with insufficient conditions for safety	Medium	Resolved
[M-05]	Batch sell matching may yield incorrect taker proceeds from <code>ctx.takerFillAmount</code>	Medium	Resolved
[L-01]	Preapproved orders skip ERC-1271 revalidation at fill time	Low	Acknowledged
[L-02]	Atomic NegRisk results are not constrained to single-winner shape	Low	Acknowledged
[L-03]	OracleAggregator accepts undocumented partial-window NegRisk requests	Low	Resolved
[L-04]	Migration snapshot can be diluted by post-prepare legacy questions	Low	Acknowledged
[L-05]	Bridge events log router as sender	Low	Resolved
[L-06]	Resolve <code>resolveCondition()</code> emits misleading oracle address in <code>ResultReported</code>	Low	Resolved
[L-07]	<code>OptimisticOracleModule.currentRequestId</code> not cleared after resolution	Low	Resolved



ID	Title	Severity	Status
[L-08]	Fee-on-transfer tokens silently underfund oracle reward	Low	Acknowledged
[L-09]	Admins can revoke peer admins	Low	Acknowledged
[L-10]	Missing sanitation check	Low	Acknowledged
[L-11]	Module replacement can fragment shared-vault liquidity for migrated position	Low	Acknowledged
[L-12]	Event-level resolution pause is dead storage for NegRisk condition	Low	Resolved
[L-13]	Admin override via <code>resolveResult</code> leaves arbitrator module stuck in <code>isActive</code>	Low	Resolved
[L-14]	<code>OracleAggregator.disputeResult</code> is blocked by pause during active dispute window	Low	Acknowledged
[L-15]	<code>InitializableRoles.addAdmin</code> contradicts the documented role management spec	Low	Resolved
[L-16]	Per-fill <code>maxFee</code> check allows excessive fees due to lack of cumulative tracking	Low	Resolved
[L-17]	<code>maxFeeRate</code> is set but not enforced during order matching validation	Low	Resolved
[L-18]	Solidity rounding allows zero taking amounts	Low	Acknowledged
[L-19]	Sequential nonces can block valid ramp approval	Low	Acknowledged
[L-20]	Collateral token fails to reconcile decimal mismatches for assets and chains	Low	Acknowledged
[L-21]	Bridged NegRisk positions may be stranded if <code>prepareConditionFromBridge</code> is first	Low	Resolved
[L-22]	<code>ChainlinkReporterModule.report</code> allows voteCount inflation due to lack of deduplication	Low	Resolved
[L-23]	Invalid outcomes return unusable legacy position IDs	Low	Resolved
[L-24]	Aggregator can mark a conflicting result resolved after the target was already resolved	Low	Resolved
[L-25]	Auto-fallback bypasses pause on the synthetic Other condition	Low	Acknowledged
[L-26]	Wrong resolver setting	Low	Acknowledged



ID	Title	Severity	Status
[L-27]	<code>PermissionedRamp</code> emits no event on witness-signed, nonce-consuming wrap/unwrap	Low	Acknowledged
[L-28]	<code>ResultReported.resolver</code> is inconsistent for contract-derived resolutions	Low	Resolved
[L-29]	ERC-1271 signature validation forwards all available gas	Low	Acknowledged
[L-30]	Compress Function Silently Destroys Dust Positions in YES Branch	Low	Acknowledged



Medium findings

[M-01] Surplus collateral can be stuck in exchange

Severity

Impact: Medium

Likelihood: Medium

Description

`Exchange._executeBatchBuyMatch` can leave collateral trapped in the `Exchange` contract when matching same-side complementary BUY orders whose combined prices are greater than one.

For a batch mint match such as `taker BUY YES` against `maker BUY NO`, the crossing check in `_validateTakerBuyMaker` only requires the combined bid prices to be at least one:

```
if (
    takerOrder.takerAmount * makerOrder.makerAmount + makerOrder.takerAmount *
    takerOrder.makerAmount
    < takerOrder.takerAmount * makerOrder.takerAmount
) revert NotCrossing();
```

This correctly rejects underfunded matches, but it also allows overfunded matches where:

```
takerPrice + makerPrice > 1
```

In that case, the taker and maker contribute more collateral than is required to split collateral into the corresponding YES/NO position set. The exchange only sends `totalMintAmount` collateral to the module and calls `_split` for that amount, but it does not refund, distribute, or account for the extra collateral.

Example:

```
Taker BUY YES: pays 50M collateral, receives 100M YES
Maker BUY NO: pays 60M collateral, receives 100M NO

Total collateral collected: 110M
Collateral required to mint 100M YES + 100M NO: 100M
Surplus left in Exchange: 10M
```

The surplus is structurally unavoidable because the operator cannot simply reduce both orders to a smaller fill amount to avoid overcollateralization: reducing the fill would compute each trader's token output as `fillAmount * takerAmount / makerAmount`, which truncates on-chain and underfills both orders. To deliver the full requested token amounts, the operator must fill both orders at their complete collateral-side amounts, making the surplus collection a necessary consequence of any matching that satisfies both orders at their stated prices.



POC

```
diff --git a/src/exchange/test/MatchOrdersBalanceFlows.t.sol b/src/exchange/test/MatchOrdersBalanceFlows.t.sol
index 1f89352..dd85e80 100644

--- a/src/exchange/test/MatchOrdersBalanceFlows.t.sol
+++ b/src/exchange/test/MatchOrdersBalanceFlows.t.sol
@@ -311,6 +311,35 @@ contract MatchOrdersBalanceFlowsTest_complementary is
MatchOrdersBalanceFlowsTest

-----*/

contract MatchOrdersBalanceFlowsTest mint is MatchOrdersBalanceFlowsTest {
+   function test_surplusCollateralRemainsWhenComplementaryBuyPricesExceedOne() public {
+       address dave = vm.addr(DAVE_PK);
+
+       dealCollateralAndApprove(bob, address(exchange), 50_000_000);
+       dealCollateralAndApprove(dave, address(exchange), 60_000_000);
+
+       // Taker bids 0.50 collateral per YES, maker bids 0.60 collateral per NO.
+       // The match mints only 100M YES + 100M NO, so only 100M collateral is
+       // burned; the extra 10M collateral remains on the exchange.
+       Order memory takerOrder = _createAndSignOrderWithSalt(bobPK, yes, 50_000_000,
100_000_000, Side.BUY, 1291);
+       Order[] memory makerOrders = new Order[](1);
+       makerOrders[0] = _createAndSignOrderWithSalt(DAVE_PK, no, 60_000_000, 100_000_000,
Side.BUY, 1292);
+
+       uint256[] memory fillAmounts = new uint256[](1);
+       fillAmounts[0] = 60_000_000;
+
+       uint256[] memory makerFees = new uint256[](1);
+
+       _runMatch(takerOrder, makerOrders, 50_000_000, fillAmounts, 0, makerFees);
+
+       assertCollateralBalance(bob, 0);
+       assertCTFBalance(bob, yes, 100_000_000);
+       assertCollateralBalance(dave, 0);
+       assertCTFBalance(dave, no, 100_000_000);
+       assertCollateralBalance(address(exchange), 10_000_000);
+       assertCTFBalance(address(exchange), yes, 0);
+       assertCTFBalance(address(exchange), no, 0);
+   }
+
+   function test_twoMakers() public {
+       address dave = vm.addr(DAVE_PK);
+       address erin = vm.addr(ERIN_PK);
```

Recommendations

Ensure the batch mint path accounts for over-collateralized buy matches and explicitly handles any surplus collateral. Detect when the combined collateral collected from buy orders exceeds the amount actually required for minting. Refund the surplus to the appropriate participant(s), or reject the match if the supplied fills would create trapped excess collateral.



[M-02] **OptimisticOracleModule** re-request reward is always zero

Severity

Impact: Medium

Likelihood: Medium

Description

`priceDisputed()` reads the reward from the OOv2 request struct via `getRequest()` and passes it to `_reRequest()`. However, the OOv2 zeroes `request.reward` before calling the `priceDisputed()` callback — it sets `request.reward = 0` as part of the `refundOnDispute()` flow triggered by `setEventBased()`. By the time `priceDisputed()` executes, `request.reward` is always `0`, and every re-request is created with no reward.

```
// src/oracle/modules/OptimisticOracleModule.sol
OptimisticOracleRequest memory req =
    optimisticOracle.getRequest(address(this), identifier, timestamp, ancillaryData);

_reRequest({
    // ...
    @> _reward: req.reward, // always 0 - OOv2 zeroes this before the callback fires
    // ...
});
```

The reward tokens are refunded to the module (it holds the balance), but they are never forwarded to the re-request. This breaks the reward recycling design described in [modules/README.md](#): "The reward cycles: module → OOv2 → (dispute refund) → module → OOv2." Re-requested conditions offer no reward to proposers, reducing the incentive to propose prices promptly and potentially stalling resolution indefinitely.

Recommendations

Store the original reward amount at request creation time (e.g., in a `conditionId → reward` mapping) and use that stored value when constructing re-requests rather than reading it from the OOv2 request struct.

[M-03] Pre-resolved neg-risk subconditions can brick outstanding OOv2 settlements

Severity

Impact: Medium

Likelihood: Medium



Description

The incremental neg-risk flow allows unresolved subconditions to be auto-resolved to `NO` once the event-level `resultsSum` reaches `RESULT_DENOMINATOR`. However, the OOv2 integration does not account for the possibility that one of those subconditions may still have a live optimistic oracle request outstanding.

This creates an idempotency mismatch between the neg-risk module and the oracle aggregator:

- `NegRiskModule.resolveCondition()` is permissionless and can finalize any remaining subcondition to `NO` once the event has already seen enough `YES` mass to imply the rest of the outcomes must be `NO`.
- `OptimisticOracleModule.priceSettled()` later finalizes the corresponding OOv2 request by calling `OracleAggregator.resolveResult()`.
- `OracleAggregator._finalizeCondition()` blindly forwards the final result into the target via `IBinaryReporter.reportResult(...)`, even if that exact target condition has already been resolved.

This issue is specific to incremental neg-risk requests, where `resultLength` is equal to `1` and each subcondition has its own OOv2 request. A realistic sequence looks like this:

1. Multiple neg-risk subconditions each create their own OOv2 requests.
2. One or more requests settle first and collectively drive `resultsSum[event]` to `RESULT_DENOMINATOR`.
3. Anyone can now call `NegRiskModule.resolveCondition()` on the remaining unresolved subconditions, which stores `NO` results directly in the module.
4. Later, one of those still-live OOv2 requests settles and triggers `OptimisticOracleModule.priceSettled()`.
5. `priceSettled()` calls `OracleAggregator.resolveResult()`, which reaches `_finalizeCondition()`.
6. `_finalizeCondition()` attempts to replay `reportResult()` into the neg-risk module for the already resolved subcondition.
7. The target reverts with `ConditionAlreadyResolved()`.

Because this revert happens inside the OOv2 settlement callback path, it bubbles all the way back into UMA's `settle()` call. The corresponding optimistic-oracle request therefore remains unsettled, which can leave the proposer/disputer bond and reward stuck in that OOv2 request.

This is not a `finalize()`-only issue. The vulnerable path for the OOv2 integration is:



```
priceSettled() -> OracleAggregator.resolveResult() ->  
OracleAggregator._finalizeCondition() -> target.reportResult()
```

The root cause is that `_finalizeCondition()` assumes the target is unresolved whenever the aggregator request itself is unresolved, but incremental neg-risk permits the target condition to be resolved externally first through `resolveCondition()`.

Recommendations

Make `_finalizeCondition()` idempotent when the target condition has already been resolved.

Before replaying `reportResult()` into the target, check whether the target condition is already resolved. If it is already resolved to the same result that the aggregator is trying to apply, treat the finalization as successful and continue. Only revert when the target has already been resolved to a conflicting outcome.

[M-04] Neg-risk migration accepts legacy events with insufficient conditions for safety

Severity

Impact: High

Likelihood: Low

Description

The neg-risk migration path does not preserve the core neg-risk invariant that an event must have at least 2 conditions.

In the native neg-risk implementation, this invariant is enforced in both `prepareEvent()` and `prepareEventFromBridge()`, which reject `_conditionCount < 2`. However, `NegRiskMigrationMixin.prepareMigrationEvent()` only checks that `NEG_RISK_ADAPTER.getQuestionCount(_legacyEventId) > 0`, and will therefore accept a legacy neg-risk event with exactly 1 condition.

This mismatch is important because the legacy adapter design itself allows one-question markets to exist. In Polymarket's legacy adapter implementation, `prepareQuestion()` can be called to add the first question to a market, while `convertPositions()` only rejects conversion when `questionCount <= 1`, which confirms that one-question markets are considered a valid legacy state even though they are not convertible yet. See [NegRiskAdapter.sol](#).

As a result, a one-condition legacy event can exist in the old system, be accepted for migration into the new neg-risk module, and then violate assumptions made by downstream event logic.



Once such an event is migrated, multiple flows break:

- `convert()` does not verify that the migrated event has at least 2 conditions. For a one-condition event, the loop that mints YES positions for "all other conditions" executes zero times, but the function still burns the caller's NO position at the end. A user converting a migrated NO position from such an event would therefore receive no YES positions in return.
- `bridgeEvent()` allows the source chain to bridge any neg-risk event whose `conditionCount > 0`. That means a migrated one-condition event can be sent into the bridge pipeline.
- On the destination chain, `_handleEventReceive()` calls `prepareEventFromBridge()`, which rejects `_conditionCount < 2`. The bridged event can therefore never be prepared successfully on the destination chain.

In effect, the migration path can import a legacy event shape that the rest of the neg-risk system does not actually support. This can lead to user fund loss in local convert flows and to unresolvable or stuck bridged markets across chains.

Recommendations

At a minimum, we need to either reject legacy neg-risk events with `conditionCount < 2` in `prepareMigrationEvent()` or configure the system to support such events.

[M-05] Batch sell matching may yield incorrect taker proceeds from `ctx.takerFillAmount`

Severity

Impact: Medium

Likelihood: Medium

Description

The sell-side batch matcher validates each maker leg independently against the taker order, but it does not settle the taker based on the batch's realized aggregate proceeds. Instead, the taker is always paid using the taker's own order ratio applied to `ctx.takerFillAmount`.

In `_executeBatchSellMatch()`, the taker's payout is computed as:

```
takerTakingAmount = ctx.takerFillAmount * takerOrder.takerAmount /  
takerOrder.makerAmount
```

This means `takerOrder.takerAmount` is effectively treated as the taker's settlement amount at full fill, even when the combination of maker legs generates a strictly better aggregate execution price.



Consider the following valid batch:

- Taker: sell 100 YES for 50 USDC
- Maker A: buy 40 YES for 24 USDC
- Maker B: sell 60 NO for 18 USDC

Both maker legs satisfy `_validateTakerSellMaker()` :

- Maker A clears as a same-token `BUY` leg
- Maker B clears as a complementary `SELL NOT` leg

The realized execution is:

1. Maker A contributes 24 USDC in exchange for 40 YES.
2. Maker B contributes 60 NO, while the taker contributes the complementary 60 YES into the merge.
3. Merging 60 YES with 60 NO returns 60 USDC.
4. Maker B is paid 18 USDC, leaving 42 USDC from the merge leg for the taker side.

So the total realized proceeds attributable to the taker's 100 YES is:

- 24 USDC from Maker A
- 42 USDC net from the merge leg
- total = 66 USDC

However, the contract does not pay the taker 66 USDC. It instead pays:

$$100 * 50 / 100 = 50 \text{ USDC}$$

which is only the taker's original limit-price amount. The remaining 16 USDC is not returned to the taker, not paid as a fee, and not otherwise accounted for. It remains stranded in the exchange contract.

As a result, favorable price improvement across a multi-leg sell batch is captured by the exchange contract rather than the taker, and the surplus becomes stuck because there is no mechanism to distribute or recover it.

Recommendations

Settle sell-side batch matches using the actual aggregate proceeds generated by maker execution, rather than paying the taker solely from their own limit price ratio.



At a minimum, we can:

- Track the actual collateral proceeds generated across all maker legs in the batch.
- Pay the taker the realized proceeds net of fees, rather than the minimum implied by `ctx.takerFillAmount`.
- If the intended behavior is to cap taker proceeds at their limit price, explicitly route the surplus somewhere intentional rather than leaving it stranded in the exchange.



Low findings

[L-01] Preapproved orders skip ERC-1271 revalidation at fill time

Description

`Exchange._validateSignature()` short-circuits whenever an order has been preapproved, regardless of whether the signature is currently valid:

```
// src/exchange/Exchange.sol
function _validateSignature(bytes32 orderHash, Order calldata order) internal view {
    if (!_isValidSignature(orderHash, order)) return;
@>    if (preapproved[orderHash]) return;
    revert InvalidSignature();
}
```

[ERC-1271](#) signatures are contract-defined and revocable, meaning that the outcome of the `isValidSignature()` check can change over time. However, the contract assumes that preapproved orders are always valid, ignoring the possibility of revocation or invalidation.

It is recommended to skip the preapproval bypass for revocable signature types and re-check ERC-1271 at fill time.

[L-02] Atomic NegRisk results are not constrained to single-winner shape

Description

`OracleAggregator._validateResult()` enforces only that the reported payout array's length matches `resultLength` and that its entries sum to at most `RESULT_DENOMINATOR`:

```
// src/oracle/OracleAggregator.sol
function _validateResult(uint8 _resultLength, uint256[] calldata _result) internal pure {
    require(_result.length == _resultLength, InvalidResultLength());

    uint256 sum;
    for (uint256 i; i < _resultLength; ++i) {
        sum += _result[i];
    }
    require(
@>        sum <= OptimisticOraclePayoutLib.RESULT_DENOMINATOR,
        InvalidResultSum()
    );
}
```



For atomic NegRisk requests (`resultLength > 1`), the documented shape — and the only shape `OptimisticOraclePayoutLib.priceToPayouts()` can produce — is winner-takes-all: exactly one entry equal to `1_000_000`, the rest `0`. The check above accepts every other distribution as well, including:

- `[250_000, 250_000, 250_000, 250_000]` (uniform "I do not know")
- `[500_000, 500_000, 0, 0]` (two co-winners)
- `[400_000, 300_000, 200_000, 100_000]` (arbitrary probability vector)
- `[250_000, 250_000, 250_000, 0]` (underpayment: total YES payout across the event sums to less than `1_000_000`)

When the aggregator fans such a result out via `_finalizeCondition()`, each subcondition's pair `[payout[i], 1_000_000 - payout[i]]` is resolved against `BaseModule._resolveCondition()`, whose only check is that the two-entry result sums to `1_000,000` — which is always trivially true. `NegRiskModule.reportResult()` additionally tracks `resultsSum[eventId]` and caps it at `1_000,000`, but this cap is satisfied by any `_validateResult`-passing input by construction, so the module does not catch the malformed shape either.

The on-chain enforcement gap is therefore: a reporter set that proposes a non-single-winner array for an atomic NegRisk event has its proposal accepted, finalized, and applied to every subcondition. Correctly long YES holders on the actual winner are underpaid (e.g. paid `0.25` instead of `1.0` under `[250k × 4]`), and YES holders on losing outcomes are overpaid.

It is recommended to enforce the documented shape inside `_validateResult()`, splitting the binary/incremental case (`resultLength == 1`), which legitimately permits an indeterminate value such as `[500_000]` produced by `OptimisticOraclePayoutLib.priceToPayouts()` for `P3_PRICE`) from the atomic NegRisk case (`resultLength > 1`), which must be winner-takes-all).

[L-03] OracleAggregator accepts undocumented partial-window NegRisk requests

Description

`OracleAggregator._isConditionInRequest()` only checks that a request's `(conditionIndex, resultLength)` pair fits inside `subconditionCount`:

```
// src/oracle/OracleAggregator.sol
function _isConditionInRequest(bytes32 _conditionId, RequestConfig storage _cfg)
internal view returns (bool) {
    uint256 conditionIndex = PositionIdLib.getConditionIndexFromCondition(_conditionId);
    return conditionIndex + _cfg.resultLength <= _cfg.subconditionCount;
}
```



The documented request shapes (binary, incremental NegRisk, atomic NegRisk) only require two modes: `resultLength == 1` (any `conditionIndex`), or `conditionIndex == 0 && resultLength == subconditionCount`. The generic check, however, also accepts contiguous "partial windows" with `1 < resultLength < subconditionCount` and `conditionIndex > 0` — e.g. `subconditionCount = 5`, `resultLength = 2`, with valid `conditionIndex` values `0`, `1`, `2`, `3`. This shape is not described in the request-type documentation.

`ChainlinkReporterModule` always reports at `eventId` (condition index 0) and is unaffected by this issue. `E0AReporterModule` is the realistic attack vector, as its authorized signer can freely choose any condition index within the valid range when constructing a report — intentionally or accidentally submitting a misaligned window and triggering the permanent deadlock.

Two consequences follow if such a request is ever configured:

1. Multiple windows on the same event can overlap on a shared `conditionIndex` (e.g., windows starting at `1` and `2` both cover index `2`). The first window to finalize calls `IBinaryReporter.reportResult` on each conditionId in its range; in `NegRiskModule._resolveCondition()` (and the binary equivalent), already-resolved conditions revert. The later overlapping window can therefore never finalize — its `_finalizeCondition` call reverts on the shared index, bricking resolution for the unique conditions that window was supposed to cover.
2. `_validateResult()` enforces `sum(_result) <= RESULT_DENOMINATOR` per request. For atomic NegRisk this is the global probability budget for the event; for partial windows it is enforced per window, so the union of payouts across non-overlapping windows can exceed `1.0`, weakening the cross-condition invariant atomic NegRisk relies on.

It is recommended to tighten `_isConditionInRequest()` (or `initializeRequest()`) to enforce the documented modes explicitly:

```
// src/oracle/OracleAggregator.sol
function _isConditionInRequest(bytes32 _conditionId, RequestConfig storage _cfg) internal
view returns (bool) {
    uint256 conditionIndex = PositionIdLib.getConditionIndexFromCondition(_conditionId);

    - return conditionIndex + _cfg.resultLength <= _cfg.subconditionCount;
+   if (_cfg.resultLength == 1) {
+       return conditionIndex < _cfg.subconditionCount;
+   }
+   return conditionIndex == 0 && _cfg.resultLength == _cfg.subconditionCount;
}
```

Alternatively, validate the shape once at `initializeRequest()` time and let `_isConditionInRequest()` keep the cheaper range check.



[L-04] Migration snapshot can be diluted by post-prepare legacy questions

Description

`NegRiskMigrationMixin.prepareMigrationEvent()` snapshots the legacy event's question count and pins it into the module's storage:

```
// src/modules/migration/NegRiskMigrationMixin.sol
function prepareMigrationEvent(bytes32 _legacyEventId) external onlyCreator {
    require(address(NEG_RISK_ADAPTER) != address(0), MigrationNotSupported());
    @> uint256 conditionCount_ = NEG_RISK_ADAPTER.getQuestionCount(_legacyEventId);
```

A NatSpec comment warns that no extra conditions should be added to the legacy event after `prepareMigrationEvent()` is called. This is enforceable for events whose oracle is controlled by Polymarket, but the legacy `NegRiskAdapter` is permissionless at the event level: a third party can create an event and set themselves as the oracle, and the oracle is the only role authorized to call `prepareQuestion()` to append new conditions. The new conditions are unregistered in `legacyConditionToConditionId` (so they are not migrated), and `_redeemPositions()` settles against the current legacy question count, diluting the migrated set's WrappedCollateral payout.

It is recommended to restrict `prepareMigrationEvent()` to legacy events whose oracle is a known, trusted address (the Polymarket UMA CTF adapter), read from the legacy adapter and checked against a constructor-configured value.

[L-05] Bridge events log router as sender

Description

`BridgeBase.bridgePositions()` and `bridgeCollateral()` emit `PositionsBridged` / `CollateralBridged` with `msg.sender` as the indexed `sender` field. When users bridge via `BridgeRouter` (the intended UX path), the router forwards the call into `IBridge`, so `msg.sender` from the bridge's perspective is the router contract — not the user that initiated the action. Every router-mediated bridge therefore emits an event whose `sender` topic is the router address, collapsing all router users into a single indistinguishable value.

The router itself emits no compensating event that records the originating EOA, so off-chain indexers, analytics, user-facing history UIs, and monitoring that rely on the indexed `sender` topic cannot attribute router-mediated bridges to the user that initiated them from logs alone.

It is recommended to emit a router-level event in `BridgeRouter._bridgePositions()` and `_bridgeCollateral()` that records `msg.sender` (the actual user) alongside the destination chain, recipient, and amount(s). Indexers can then join router events to bridge events via transaction hash. Alternatively, extend `IBridge.bridgePositions()` / `bridgeCollateral()` with an explicit `initiator` parameter that the router populates with `msg.sender` and the bridge emits in the event.



[L-06] Resolve `resolveCondition()` emits misleading oracle address in `ResultReported`

Description

`NegRiskModule.resolveCondition()` is called by anyone once YES results sum to `RESULT_DENOMINATOR`. It emits `ResultReported(msg.sender, _conditionId, result_)`, but `msg.sender` here is the external caller of `resolveCondition()` — not any oracle. The event's first parameter is documented as "The oracle that reported," so off-chain indexers and monitoring tools that consume this event will record an arbitrary address as the oracle for this condition.

Consider emitting `address(this)` as a sentinel value for the oracle address, which is consistent with how `OracleAggregator.finalize()` emits `RequestResolved` with `address(this)` as the resolver to signal that the resolution was triggered by the module itself rather than a reporting oracle.

```
function resolveCondition(bytes32 _conditionId) external {  
    // ...  
  
    - emit ResultReported(msg.sender, _conditionId, result_);  
    + emit ResultReported(address(this), _conditionId, result_);  
}
```

[L-07] `OptimisticOracleModule.currentRequestId` not cleared after resolution

Description

`priceSettled()` does not clear `currentRequestId[conditionId]` after calling `resolveResult()`. Since `forceRerequest()` only gates on `currentRequestId[_conditionId] != bytes32(0)`, an admin can call it on a condition that has already been finalized, creating a new OOV2 request against an already resolved aggregator condition.

The new request becomes permanently stuck. OOV2's `_settle()` sets `request.settled = true` before invoking `priceSettled()`, which then calls `resolveResult()` and reverts with `RequestAlreadyResolved`. The revert rolls back the entire `settle()` transaction, but the `settled` flag persists, making the request unseizable. Any proposer or disputer who bonded on it cannot recover their funds.

It is recommended to delete `currentRequestId[conditionId]` in `priceSettled()` immediately after `resolveResult()` succeeds.



```
// src/oracle/modules/OptimisticOracleModule.sol
uint256[] memory result = OptimisticOraclePayoutLib.priceToPayouts(_price,
outcomeCount);
IOracleAggregator(aggregator).resolveResult(conditionId, result);
+ delete currentRequestId[conditionId];
```

[L-08] Fee-on-transfer tokens silently underfund oracle reward

Description

`OptimisticOracleModule.initializeRequest()` transfers `_reward` tokens from the caller and passes that same `_reward` value to the Optimistic Oracle. If `_bondToken` is a fee-on-transfer token, the contract receives less than `_reward` but approves the full amount to the oracle, pulling the shortfall from whatever residual balance the module holds. The oracle ends up with the correct nominal reward credited on its side, but the module's token balance is silently drained by the fee on every request. Over time, this drains any accumulated surplus and eventually causes `initializeRequest()` to revert once the balance is exhausted.

It is recommended to measure the actual received amount — balance before minus balance after the `safeTransferFrom` — and use that figure when configuring the oracle request rather than `_reward` directly, or to document that fee-on-transfer tokens are not supported.

[L-09] Admins can revoke peer admins

Description

Several admin role-management helpers allow any current admin to remove other admins. For example, `Roles.removeAdmin()` is guarded by `onlyAdmin`, even though `Roles.addAdmin()` is owner-only:

```
// src/auth/Roles.sol
function addAdmin(address _admin) external onlyOwner {
    _grantRoles(_admin, _ROLE_0);
}

@> function removeAdmin(address _admin) external onlyAdmin {
    _removeRoles(_admin, _ROLE_0);
}
```

The same peer-admin removal pattern appears in `InitializableRoles`, `Auth`, `CollateralOnramp`, `CollateralOfframp`, `PermissionedRamp`, and `CtfCollateralAdapter`.

The removal of an admin will most likely be used when the admin is not trusted anymore, so allowing any admin to perform this action could lead to the malicious admin removing all other admins before they remove him.

It is recommended to restrict admin-role addition and removal to the owner, or explicitly document and consistently apply a peer-admin governance model across all contracts.



[L-10] Missing sanitation check

Description

- `ChainlinkReporterModule.initializeReporterModule()` should validate that the request is binary-compatible. The module stores one event-level candle config and `report()` always returns a one-element YES/NO result, so it should not be silently initialized for unsupported request shapes.
- `ChainlinkReporterModule.initializeReporterModule()` should validate the candle timing parameters. In particular, reject `duration == 0` if zero-length windows are not intended, and reject stale initialization where `startTimestamp + duration <= block.timestamp` if requests must be configured before the candle closes.
- `ChainlinkReporterModule.report()` should enforce that the candle window has ended. Currently, reporting can succeed before the window ends if both DataStore price slots have already been written. There is a `WindowNotEnded` error defined but not used.
- `Exchange` order validation should bound `order.makerAmount` to `type(uint248).max`. The `OrderStatus` slot packs `{bool filled : 8 bits, uint248 remaining : 248 bits}` and `_storeOrderStatus()` writes via `shl(8, remaining)` with no overflow check, so a `makerAmount` exceeding 248 bits would silently truncate the high bits and could corrupt the `filled` flag. The current validation only checks `makerAmount != 0`. The cap is economically implausible to hit, but the safety should be explicit rather than implicit.
- `OracleAggregator.initializeRequest()` should validate that `_params.eventId` has the canonical event-id shape (bottom 88 bits zero). Config is stored under `_params.eventId` as-is, but downstream lookups strip those low bits via `getEventIdFromCondition()`. A malformed eventId (e.g. a conditionId or any value with non-zero reserved/conditionIndex/outcomeIndex bits) would make the config permanently unreachable while also freezing the storage slot, since `cfg.targetContract != address(0)` blocks re-initialization.

It is recommended to add sanitation checks in the mentioned functions to prevent unexpected behavior.

[L-11] Module replacement can fragment shared-vault liquidity for migrated position

Description

The protocol routes historical positions by `moduleId`, not by the original module contract that prepared or migrated them. If a migration-enabled module is replaced, user-facing operations for those historical positions begin routing through the new module immediately, even though the legacy backing for some migrated positions may still be held and realized by the old module.



This becomes problematic because migrated legacy inventory is custodied by the migration module itself. In `BaseMigrationMixin`, legacy CTF positions are transferred into `address(this)` during migration:

- `_legacyConditionalTokens().safeBatchTransferFrom(_from, address(this), legacyPositionIds, _amounts, "")`

and only later, when that legacy value is monetized, the old module settles it into the shared vault through:

- `_redeemPositions(...)`
- `_settleLegacyCollateralToVault()`

At the same time, the replacement module can immediately service standard merge flows for the same historical position IDs because routing now resolves to the new module by `moduleId`.

This creates a liquidity-fragmentation scenario before any results are reported:

- Users with a complete set on the replacement module can call `merge()`
- `merge()` does not require the market to be resolved
- The replacement module mints `pUSD` for the user
- The user can then unwrap against the shared vault

Ordinarily, a complete-set merge should preserve liquidity alignment for the system: collapsing a fully backed position set is expected to correspond to backing being available for the shared vault. However, after a module replacement, that expectation can break: the migrated backing may still be trapped in the old module's legacy settlement path and may not have reached the shared vault yet.

As a result:

- Unresolved migrated positions can extract shared vault liquidity through `merge()` on the new module.
- The corresponding legacy-backed value may still remain unrealized on the old module.
- Unrelated resolved positions that rely on the same shared vault may later be unable to unwrap or redeem underlying assets because that liquidity has already been consumed.
- Liquidity only normalizes once the old module eventually realizes and settles its legacy inventory.

This creates a real denial-of-service condition for unrelated exits. Even though complete-set merges would ordinarily imply that equivalent backing is available to the system, module replacement allows those merges to drain the shared vault before the old module has delivered the corresponding legacy value. The core issue is that module replacement changes the execution endpoint for historical positions without providing a mechanism to transfer or synchronize the old module's backing before complete-set merges resume on the new module.



Recommendations

Introduce a dedicated backing transfer or settlement synchronization procedure before replacing a migration-enabled module.

[L-12] Event-level resolution pause is dead storage for NegRisk condition

Description

`OracleModule.pause(bytes32 _id)` is documented to pause resolution "for a specific condition or event". But nothing in `NegRiskModule` ever reads `resolutionPausedAt[eventId]`:

- `reportResult` uses the `onlyOracle(_conditionId)` modifier, which only checks `resolutionPausedAt[_conditionId]`.
- `resolveCondition` (the permissionless NO-finalizer) also only checks `resolutionPausedAt[_conditionId]`:

```
// NegRiskModule.sol
function resolveCondition(bytes32 _conditionId) external {
    require(oraclePausedAt[oracle[_conditionId]] == 0, OracleIsPaused());
    require(resolutionPausedAt[_conditionId] == 0, ResolutionIsPaused());
    bytes32 eventId_ = PositionIdLib.getEventIdFromCondition(_conditionId);
    ...
}
```

An admin calling `pause(eventId)` to freeze an entire event therefore has no effect: both `reportResult` and `resolveCondition` continue to work on individual conditions in that event. To actually freeze the event, the admin must pause every remaining `conditionId` individually.

Add event-level pause checks on both paths (and/or extend the `onlyOracle` modifier to also check the event ID), e.g.:

```
function resolveCondition(bytes32 _conditionId) external {
    require(oraclePausedAt[oracle[_conditionId]] == 0, OracleIsPaused());
    require(resolutionPausedAt[_conditionId] == 0, ResolutionIsPaused());
    bytes32 eventId_ = PositionIdLib.getEventIdFromCondition(_conditionId);
+   require(resolutionPausedAt[eventId_] == 0, ResolutionIsPaused());
    ...
}
```



[L-13] Admin override via `resolveResult` leaves arbitrator module stuck in `isActive`

Description

`OracleAggregator.resolveResult` can finalize a request directly, callable by either the configured arbitrator module or an administrator, with the only status guard being `state.status != Resolved`:

```
// src/oracle/OracleAggregator.sol
function resolveResult(bytes32 _requestId, uint256[] calldata _result) external whenUnpaused
{
    ...
    require(state.status != ResolutionStatus.Resolved, RequestAlreadyResolved());

    bool isArb = msg.sender == cfg.arbitratorModule;
    bool isAdmin = hasAllRoles(msg.sender, _ROLE_0);
    require(isArb || isAdmin, NotAuthorized());
    ...
}
```

This means an admin can finalize a condition that is in `ArbitrationRequested`, i.e., after the aggregator has already called `IArbitratorModule.onArbitrationTriggered` on the configured arbitrator module. When that happens, the aggregator flips its own status to `Resolved`, but the arbitrator module is never told its case ended. The `IArbitratorModule.getArbitrationState` view keeps returning `isActive = true` for that request indefinitely, which any off-chain consumer or downstream integration inspecting the arbitrator sees as a live case.

The arbitrator-path call (arbitrator module calling `resolveResult` itself) can clean up its own state inline, so the issue is specific to the admin override path.

Recommendations

Extend `IArbitratorModule` with an `onArbitrationResolved(bytes32 requestId)` hook and call it from `resolveResult` when `state.status == ArbitrationRequested` and the caller is not the arbitrator itself, so that the arbitrator module can clear its `isActive` flag.

[L-14] `OracleAggregator.disputeResult` is blocked by pause during active dispute window

Description

`OracleAggregator.disputeResult` is gated by `whenUnpaused`, but the dispute deadline (`state.disputeWindowEnd`) is set at proposal time as `block.timestamp + cfg.livenessWindow` and is never adjusted for paused intervals:



```
// src/oracle/OracleAggregator.sol
function disputeResult(bytes32 _requestId) external whenUnpaused {
    ...
    require(block.timestamp < state.disputeWindowEnd, DisputeWindowExpired());
    ...
}
```

If the aggregator is paused after a proposal, the chance to dispute keeps shrinking while disputers are locked out. Once the pause ends, if `block.timestamp >= state.disputeWindowEnd`, no dispute can ever be raised and anyone can immediately call `finalize` on the unchallenged proposal.

A correct fix is non-trivial. It must extend `disputeWindowEnd` only for requests whose window was actively open during a pause, and only by the overlap between each pause interval and that specific window. Additionally, `finalize`'s `block.timestamp >= state.disputeWindowEnd` check must use the same effective deadline as `disputeResult`.

Alternatively, when unpausing, extend all dispute window end times that have not yet expired by the full configured liveness window duration. This simpler approach ensures every disputer always has at least a full liveness window after an unpause, at the cost of potentially over-extending windows that had nearly elapsed before the pause.

[L-15] `InitializableRoles.addAdmin` contradicts the documented role management spec

Description

`InitializableRoles` is the role base used by `PositionManager`. Its `addAdmin` is gated by the admin role itself:

```
// src/auth/InitializableRoles.sol
function addAdmin(address _admin) external onlyAdmin {
    _grantRoles(_admin, _ROLE_0);
}
```

This contradicts `docs/auth.md`, which specifies `addAdmin = onlyOwner` for `Roles.sol` and states that `InitializableRoles` role management is identical to `Roles`:

Role management functions are identical to `Roles` except `addBridge` and `removeBridge` are absent. — `docs/auth.md`

Accordingly, `Roles.addAdmin` is `onlyOwner` and so are a few other admin-gated contracts (`Exchange`, `oracle/mixins/Auth`, `LzBridge`, `CcipBridge`, etc.).

`InitializableRoles` is the sole outlier where documentation does not match implementation, and it is the one protecting `PositionManager`.



The practical consequence is an admin self-perpetuation primitive: a rogue or compromised admin can call `addAdmin(attacker)`, after which owner-led recovery must remove every attacker-installed admin atomically, because any remaining attacker admin can immediately re-grant the role. Under the documented `onlyOwner` gating, this race does not exist.

An additional deployment concern: if an inheriting contract fails to grant at least one admin during initialization, the custom `addAdmin()` path becomes unusable until the owner calls the inherited `grantRoles()` function directly — which may not be exposed in standard operational tooling or anticipated by integrators who expect `addAdmin()` to work as documented.

Bring `InitializableRoles.addAdmin` in line with `Roles` and `docs/auth.md`:

```
- function addAdmin(address _admin) external onlyAdmin {  
+ function addAdmin(address _admin) external onlyOwner {  
    _grantRoles(_admin, _ROLE_0);  
}
```

If the current behavior is actually intended, update `docs/auth.md` instead to reflect that `InitializableRoles` deviates from `Roles` on this function.

[L-16] Per-fill `maxFee` check allows excessive fees due to lack of cumulative tracking

Description

The fee check in `_validateOrder` runs independently per fill:

```
// Exchange.sol  
if (fee > order.maxFee) revert FeeExceedsMaxFee();
```

Each time an order is partially filled (via separate `matchOrders` calls), the operator can charge up to `order.maxFee`. There is no cumulative fee tracking across fills.

The per-order `maxFee` field is naturally understood as "the maximum fee this order will ever pay." The current behavior turns it into "the maximum fee per fill," which is a weaker guarantee that grows linearly with the number of partial fills. While the trusted operator model partially mitigates this in practice, users signing orders have no on-chain guarantee that their total fees will be bounded.

Track cumulative fees per order hash in the `OrderStatus` packed storage and check the running total against `maxFee`.



[L-17] `maxFeeRate` is set but not enforced during order matching validation

Description

`Exchange.maxFeeRate` is initialized to `200` (2%) in the constructor, has an admin setter `setMaxFeeRate`, and is publicly readable. However, it is never referenced in any validation logic during `matchOrders` or any of its internal call chains.

```
// stored but never read during matching
uint256 public maxFeeRate;

// initialized
maxFeeRate = 200;

// setter exists
function setMaxFeeRate(uint256 _maxFeeRate) external onlyRoles(ADMIN_ROLE) {
    maxFeeRate = _maxFeeRate;
}
```

The only on-chain fee enforcement is the per-order absolute cap:

```
if (fee > order.maxFee) revert FeeExceedsMaxFee();
```

This means the global rate limit that `maxFeeRate` implies is never enforced on-chain. The operator can charge any absolute fee up to each order's `maxFee` regardless of trade size and regardless of the configured `maxFeeRate`. Integrators reading `maxFeeRate` may incorrectly assume it is enforced.

Either enforce `maxFeeRate` during matching or remove the variable to avoid misleading integrators.

[L-18] Solidity rounding allows zero taking amounts

Description

`Exchange._calculateTakingAmount` floors the counter-asset amount for every partial fill:

```
function _calculateTakingAmount(uint256 makingAmount, uint256 makerAmount, uint256
takerAmount)
    internal
    pure
    returns (uint256)
{
    return makingAmount * takerAmount / makerAmount;
}
```

For a maker `SELL` order, `makingAmount` is the filled position-token amount and `takerAmount` is the collateral the maker expects for the full order. If a partial fill is small enough that:



```
makingAmount * takerAmount < makerAmount
```

then `_calculateTakingAmount` returns zero. The maker's remaining position amount is still reduced, but the maker receives no collateral for that fill.

Example with 6-decimal assets:

Victim SELL order:

- Sells 1,000 YES
- Wants 10 USDC
- Price: 0.01 USDC per YES

```
makerAmount = 1,000_000000
```

```
takerAmount = 10_000000
```

An attacker can fill a dust amount:

```
makerFillAmount = 99 base units = 0.000099 YES
```

```
takingAmount = 99 * 10_000000 / 1,000_000000  
              = 0
```

The victim loses `0.000099 YES` and receives `0` collateral. The attacker receives the position tokens without paying collateral.

The maximum zero-collateral fill size for a single match is:

```
floor((makerAmount - 1) / takerAmount)
```

For normal 6-decimal markets, this usually limits the practical impact to dust. For example:

```
1,000 YES sold for 10 USDC -> max free fill: 0.000099 YES  
1,000 YES sold for 100 USDC -> max free fill: 0.000009 YES  
1,000 YES sold for 500 USDC -> max free fill: 0.000001 YES
```

POC

```
// SPDX-License-Identifier: MIT  
pragma solidity ^0.8.34;  
  
import { BaseExchangeTest } from "./BaseExchangeTest.sol";  
import { Order, Side } from "src/exchange/OrderStructs.sol";  
  
contract PartialFillRoundingPoC is BaseExchangeTest {  
    uint256 internal constant UNIT = 1_000_000;  
  
    function _matchSingleMaker(  
        Order memory takerOrder,  
        Order memory makerOrder,  
        uint256 takerFillAmount,  
        uint256 makerFillAmount  
    ) internal {  
        Order[] memory makerOrders = new Order[](1);
```



```
makerOrders[0] = makerOrder;

uint256[] memory fillAmounts = new uint256[](1);
fillAmounts[0] = makerFillAmount;

uint256[] memory makerFees = new uint256[](1);

vm.prank(admin);
exchange.matchOrders(takerOrder, makerOrders, takerFillAmount, fillAmounts,
conditionId, 0, makerFees);
}

function test_BUG_realisticDustFillRoundsSellOrderProceedsToZero() public {
    uint256 victimPositionAmount = 1_000 * UNIT; // 1,000 YES
    uint256 victimExpectedCollateral = 10 * UNIT; // 10 USDC, or 0.01 USDC per YES
    uint256 freeFillAmount = 99; // 0.000099 YES

    dealCollateralAndApprove(bob, address(exchange), 1);
    dealOutcomeTokensAndApprove(carla, address(exchange), victimPositionAmount);

    Order memory attackerBuyOrder = _createAndSignOrderWithSalt(bobPK, yes, 1, 1,
Side.BUY, 9101);
    Order memory victimSellOrder =
        _createAndSignOrderWithSalt(carlaPK, yes, victimPositionAmount,
victimExpectedCollateral, Side.SELL, 9102);

    _matchSingleMaker(attackerBuyOrder, victimSellOrder, 1, freeFillAmount);

    assertCTFBalance(carla, yes, victimPositionAmount - freeFillAmount);
    assertCollateralBalance(carla, 0);
    assertCTFBalance(bob, yes, freeFillAmount);
    assertCollateralBalance(bob, 1);

    (bool victimFilled, uint248 victimRemaining) =
exchange.orderStatus(exchange.hashOrder(victimSellOrder));
    assertFalse(victimFilled);
    assertEq(victimRemaining, victimPositionAmount - freeFillAmount);

    (bool attackerFilled, uint248 attackerRemaining) =
exchange.orderStatus(exchange.hashOrder(attackerBuyOrder));
    assertFalse(attackerFilled);
    assertEq(attackerRemaining, 1);
}
}
```

Recommendations

Reject non-zero fills that produce zero taking amount:

```
uint256 takingAmount = _calculateTakingAmount(fillAmount, order.makerAmount,
order.takerAmount);
require(takingAmount != 0, ZeroTakingAmount());
```



The zero-taking-amount guard must also be applied in the complementary fast path functions: `_executeComplementaryBuyFastPath`, `_executeComplementarySellFastPath`, and `_executeComplementaryZeroFeeSellFastPath`. In these paths, the taker order's remaining is only decremented by the accumulated `takerMakingAmount` — so a fill that rounds to zero contributes nothing, leaving the taker order's remaining unchanged and the same signed taker order indefinitely reusable across future matches while each match consumes maker liquidity. Alternatively, instead of rejecting zero-taking fills, use ceiling division to round the taker-side payment up so the maker always receives at least 1 unit of collateral for any nonzero fill. This preserves partial-fill semantics while eliminating the possibility of makers being shortchanged.

[L-19] Sequential nonces can block valid ramp approval

Description

`PermissionedRamp` uses a single monotonically increasing nonce per sender for both permissioned wraps and unwraps. As a result, valid witness signatures for a sender must be executed exactly in nonce order, even if the witness service has already authorized multiple independent operations for that sender. If an earlier approval is not submitted, is intentionally abandoned, or expires, later approvals remain unusable until the sender obtains and executes another signature for the current nonce.

```
// src/collateral/PermissionedRamp.sol
function _validateSignature(
    bytes32 _typehash,
    address _asset,
    address _to,
    uint256 _amount,
    uint256 _nonce,
    uint256 _deadline,
    bytes calldata _signature
) internal {
    require(block.timestamp <= _deadline, ExpiredDeadline());
    @> require(_nonce == nonces[msg.sender]++, InvalidNonce());
```

This can cause avoidable denial of service for permissioned ramp operations, especially when users request multiple signatures or the off-chain signer issues replacement approvals. It also makes cancellation semantics ambiguous: a later signed approval may become executable after an older nonce is consumed, as long as the later approval has not expired.

Recommendations

Use unordered per-sender nonces instead of a strict incrementing nonce. For example, include a witness-selected nonce in the EIP-712 payload and track consumed nonces with `mapping(address => mapping(uint256 => bool)) usedNonces`, reverting only when a nonce has already been used. This allows independently authorized approvals to execute in any order while preserving replay protection and making cancellation depend on explicit expiry or signer policy. Alternatively, if switching to unordered nonces is not feasible, add an explicit



`incrementNonce()` function that lets a sender advance their current nonce without performing a ramp operation. This allows invalidation of any pending approval for the current nonce while preserving the sequential nonce model and its associated simplicity.

[L-20] Collateral token fails to reconcile decimal mismatches for assets and chains

Description

The collateral system assumes that all supported collateral assets use 6 decimals, but this assumption is not enforced or normalized throughout the flow.

`CollateralToken.sol` hardcodes `pUSD` to 6 decimals, while `wrap()` and `unwrap()` mint and burn collateral 1:1 against the raw `_amount` of the selected underlying asset. In addition, `BridgeBase.sol` forwards the same raw collateral amount cross-chain and mints that exact amount on the destination chain without including any decimal metadata or performing any source/destination decimal conversion.

This becomes especially dangerous in the concrete case of BNB Chain, where Circle's `FiatTokenV2_2` deployment for USDC uses 18 decimals instead of the more common 6. If Polymarket collateral were ever deployed against a BNB Chain-style USDC configuration, the protocol would treat the raw token amount as though it were already denominated in 6-decimal units.

For example, assume a user bridges or wraps 20 USDC on BNB Chain. Because BNB Chain USDC uses 18 decimals, the raw amount moved through the system is `20e18`. However, `pUSD` is fixed at 6 decimals, and the current implementation mints and bridges collateral 1:1 using that raw amount. As a result, the recipient would receive `20e18` units of `pUSD`, which corresponds to `20e12` tokens instead of 20 tokens.

This can lead to severe accounting failures:

- Users can be over-minted collateral relative to the underlying assets deposited.
- Cross-chain collateral transfers can mint an incorrect amount on the destination chain.
- Unwrap flows can become inconsistent because the protocol treats collateral and underlying assets as if they always share the same decimal precision.
- The protocol may become undercollateralized or otherwise insolvent if unsupported decimal configurations are ever deployed.

At present, this appears to be a latent issue because the currently configured collateral assets in the codebase and test setup use 6 decimals. However, the system does not defend itself against a future deployment or expansion that introduces a collateral asset or chain with a different decimal configuration.



Recommendations

Explicitly enforce a single supported decimal precision for all collateral assets at deployment time, or add decimal normalization logic anywhere collateral amounts cross an asset or chain boundary.

[L-21] Bridged NegRisk positions may be stranded if `prepareConditionFromBridge` is first

Description

`NegRiskModule` does not override the inherited `BaseModule.prepareConditionFromBridge`, which sets `oracle[conditionId] = bridge` without touching `conditionCount[eventId]`:

```
// src/modules/abstract/BaseModule.sol
function prepareConditionFromBridge(bytes32 _conditionId) external virtual onlyBridge {
    if (isConditionPrepared(_conditionId)) return;
    oracle[_conditionId] = msg.sender;
    emit ConditionPrepared(_conditionId, msg.sender, "");
}
```

Two permissionless paths in `BridgeBase` reach this function with NegRisk condition IDs on the destination chain:

1. `bridgeCondition(dst, negRiskConditionId, ...)` → `_handleConditionReceive` → `prepareConditionFromBridge`.
2. `bridgePositions(dst, [negRiskPositionIds...], ...)` → `_handlePositionsReceive` calls `prepareConditionFromBridge` before `mintFromBridge` for every bridged position (`BridgeBase.sol:554`).

Both entry points are `external payable` without access control, so any user can put any NegRisk event into the half-prepared state (`oracle != 0`, `conditionCount[eventId] == 0`) on any spoke.

Once in this state, every path that reads `conditionCount` reverts with `EventNotPrepared`: `reportResult`, `resolveCondition`, `horizontalSplit`, `horizontalMerge`, `convert`. In particular, incoming `RESULT` bridge messages fail at `_handleResultReceive` → `reportResult` and land in the transport's failed-message queue until the spoke state is repaired, so NegRisk positions minted on the spoke are not redeemable until an `EVENT` message is delivered and the `RESULT` is re-executed. `prepareEventFromBridge` can overwrite the state and recover, but only once an `EVENT` message arrives — someone must notice and call `bridgeEvent` from the hub.



Recommendations

Override `prepareConditionFromBridge` on `NegRiskModule` so that misrouted condition/position bridges fail loudly (and land in the transport retry queue) instead of silently corrupting state:

```
// src/modules/NegRiskModule.sol
function prepareConditionFromBridge(bytes32 _conditionId) external override onlyBridge {
    bytes32 eventId_ = PositionIdLib.getEventIdFromCondition(_conditionId);
    require(conditionCount[eventId_] > 0, EventNotPrepared());
    // oracle is already set by prepareEventFromBridge; nothing else to do here
}
```

A plain `revert` override would also brick legitimate NegRisk position bridges (every bridged position flows through `_handlePositionsReceive` → `prepareConditionFromBridge`), so the no-op-when-prepared form is required.

Additionally, reject the misrouted entry at the source in `BridgeBase.bridgeCondition` so the failure surfaces up front instead of after bridge fees have been paid:

```
// src/bridge/abstract/BridgeBase.sol - bridgeCondition
require(moduleId != ModuleIds.NEGRISK, UseBridgeEventForNegRisk());
```

Document the resulting invariant: `bridgeEvent(dst, eventId)` must be executed once per spoke before any NegRisk condition or position can be bridged to that spoke.

[L-22] `ChainlinkReporterModule.report` allows voteCount inflation due to lack of deduplication

Description

`ChainlinkReporterModule.report()` is publicly callable with no per-request deduplication. The developer comment states "this function allows reporting multiple times, since registerReport on the aggregator is idempotent for these types of markets" — but the aggregator is NOT idempotent. Each call increments `voteCount` in `OracleAggregator.reportResult`:

```
// OracleAggregator.reportResult
uint256 newCount = voteCount[voteKey] + 1;
voteCount[voteKey] = newCount;

if (newCount >= cfg.reporterThreshold && state.proposedResultHash == bytes32(0)) {
    state.proposedResultHash = resultHash;
    // ...
}
```

If a market is configured with multiple reporter modules and `reporterThreshold > 1` to require multi-source consensus, anyone can call `ChainlinkReporterModule.report()` N times to unilaterally reach the threshold from a single data source, bypassing the independent confirmation from other reporters.



The `E0AReporterModule` correctly prevents this via its internal `hasReported` mapping — the `ChainlinkReporterModule` has no equivalent guard.

Once the inflated vote count crosses the threshold and the liveness window expires, the proposal can be finalized normally — meaning the bypass not only corrupts the multi-source consensus assumption but also results in on-chain finalization of a result backed by a single data source, with no remaining obstacle for a disputer to intervene.

Recommendations

Add per-request deduplication to `ChainlinkReporterModule.report()` :

```
mapping(bytes32 => bool) public hasReported;

function report(bytes32 _requestId) external whenUnpaused {
    bytes32 eventId = _getEventIdForCondition(_requestId);
    require(!hasReported[eventId], AlreadyReported());
    hasReported[eventId] = true;
    // ...
}
```

[L-23] Invalid outcomes return unusable legacy position IDs

`BinaryMigrationMixin.getLegacyPositionId()` documents `_outcomeIndex` as `0` or `1`, but does not enforce that bound before converting it to a legacy CTF index set. As a result, callers can pass invalid outcomes and receive a deterministic position ID for an invalid single-bit index set instead of an explicit failure or zero value.

```
// src/modules/migration/BinaryMigrationMixin.sol
function getLegacyPositionId(bytes32 _conditionId, uint256 _outcomeIndex) public view
returns (uint256) {
    bytes32 legacyConditionId_ = legacyConditionId[ConditionIdLib.from(_conditionId)];
    if (legacyConditionId_ == bytes32(0)) return 0;
    @> return CTHelpers.getPositionId(
        address(USDCE), CTHelpers.getCollectionId(bytes32(0), legacyConditionId_,
        _outcomeIndex + 1)
    );
}
```

This can mislead integrators, scripts, or off-chain reconciliation tooling that rely on it to derive canonical legacy YES/NO position IDs.

It is recommended to validate `_outcomeIndex < 2` in `getLegacyPositionId()` and revert with `InvalidOutcomeIndex()` for invalid inputs.



[L-24] Aggregator can mark a conflicting result resolved after the target was already resolved

Affected files:

- `src/modules/BinaryModule.sol:56-61`
- `src/modules/NegRiskModule.sol:121-126`

Because of the fix for prior report [M-03] **Pre-resolved neg-risk subconditions can brick outstanding OOV2 settlements** that was introduced in commit `0c9a6470` (`feat: remove prepare condition (#182)`) and is still present in latest HEAD `8009ad9` . That fix made `BinaryModule.reportResult()` and `NegRiskModule.reportResult()` return early whenever `result[_conditionId].length > 0` , preventing later OOV2 settlements from reverting after a neg risk subcondition was already auto resolved.

So the fix is currently incomplete because it made already resolved targets an unconditional no op instead of the M-03 recommendation: treat the replay as successful only when the existing target result matches the result being finalized, and reject conflicting outcomes. As a result, `OracleAggregator.resolveResult()` or `finalize()` can store a new `proposedResultHash` , mark the request `Resolved` , and emit `RequestResolved` even though the target module ignored the report and kept a different payout vector. This conflicts with the oracle docs, which describe `Resolved` as the final result being reported to the target contract.

The latest commit itself (`8009ad9`) did not introduce this behavior; it retained it.

Recommendation

Make idempotency same result only. Before treating an already resolved target as a successful no op, compare the existing module result with the binary result the aggregator is trying to apply. Return only on an exact match, or revert or record a conflict state. If bridge result replay support is required, keep that replay path idempotent only for identical results, not for conflicting resolver outputs.

[L-25] Auto-fallback bypasses pause on the synthetic Other condition

`OracleModule.pauseResolution(_id)` is the admin's per-condition kill switch: it sets `resolutionPausedAt[_id]` , and the `onlyResolver(_id)` modifier reverts `ResolutionIsPaused()` for any direct `reportResult(_id, ...)` against a paused condition. The bridge/resolver-initiated path honors this correctly.

The auto-fallback inside `_finalizeNegriskResolution` does not. When the final real condition resolves — i.e. `conditionsResolved_ == conditionCount_` or `resultsSum_ == RESULT_DENOMINATOR` — the synthetic "Other" result is written through `_storeResult` directly, with no consultation of `resolutionPausedAt[fallbackConditionId]` :



```
// src/modules/NegRiskModule.sol _finalizeNegriskResolution()
ConditionId fallbackConditionId = eventId_.computeConditionId(conditionCount_);
if (
    (conditionsResolved_ == conditionCount_ || resultsSum_ == RESULT_DENOMINATOR)
    && result[fallbackConditionId].length == 0
) {
    uint256[] memory fallbackResult = new uint256[](2);
    fallbackResult[0] = RESULT_DENOMINATOR - resultsSum_;
    fallbackResult[1] = resultsSum_;
    @> _storeResult(fallbackConditionId, fallbackResult); // no resolutionPausedAt
    check
}
```

As a result, an admin who pauses only the synthetic Other condition gets a false sense of safety: the next real-condition report auto-derives and stores Other anyway, and users can immediately redeem `YES(Other)` / `NO(Other)`. This is an inconsistency between the bridge path (pause-aware) and the auto-fallback path (pause-blind). The current mitigation is to pause a real condition instead, which does block finalization through `onlyResolver`, but in that case the "Other" condition could still be bridged and resolved (unless the admin calls `OracleModule.pauseResolution(_id)` on the "Other" condition directly), so it's still inconsistent behavior.

Either gate the auto-fallback write with `resolutionPausedAt[fallbackConditionId] == 0`, or document in the `pauseResolution()` NatSpec that pausing the synthetic Other index is a no-op against auto-fallback.

Recommendations

Specifically, add the following check inside `_finalizeNegriskResolution()` immediately before the `_storeResult(fallbackConditionId, fallbackResult)` call:

```
require(resolutionPausedAt[fallbackConditionId] == 0, ResolutionIsPaused());
```

Client Commentary

Resolution controls should be applied to the relevant real conditions or resolver. The synthetic Other result is contract-derived, and pausing the triggering real condition prevents derivation through that path.

[L-26] Wrong resolver setting

`RequestResolved` is the single event for both resolution paths. Its NatSpec defines the indexed `resolver` field as "The address that triggered the resolution". The two emit sites disagree:

```
// resolveResult (arbitrator/admin path)
emit RequestResolved(_requestId, resultHash,
    msg.sender); // resolver = actual caller

// finalize (permissionless liveness path)
emit RequestResolved(_requestId, state.proposedResultHash,
    address(this)); // resolver = the aggregator itself
```



`finalize` is permissionless (anyone can call it after the liveness window). The address that actually triggered the resolution is `msg.sender`, but the event reports the aggregator's own address. Because `resolver` is indexed, consumers filtering `RequestResolved` by an EOA/relayer will never match any finalize-path resolution.

Indexed-field semantics do not match the documented meaning, and filtering resolutions by who actually triggered them is broken for the finalize path.

Emit `msg.sender` as the resolver in `finalize()` path, same as in `resolveResult()` path.

Fix Commentary

Documentation adjusted.

[L-27] `PermissionedRamp` emits no event on witness-signed, nonce-consuming wrap/unwrap

`PermissionedRamp.wrap()` / `unwrap()` consume a per-sender nonce, yet the contract defines no events and emits nothing on these paths.

```
function wrap(address _asset, address _to, uint256 _amount, uint256 _nonce, uint256
_deadline, bytes calldata _signature)
    external onlyUnpaused(_asset)
{
    _validateSignature({ ... });
    _asset.safeTransferFrom(msg.sender, COLLATERAL_TOKEN, _amount);
    CollateralToken(COLLATERAL_TOKEN).wrap(_asset, _to, _amount);
}

// ...

function _validateSignature(
    address _asset,
    address _to,
    uint256 _amount,
    uint256 _nonce,
    uint256 _deadline,
    bytes calldata _signature
) internal {
    require(block.timestamp <= _deadline, ExpiredDeadline());
    @> require(_nonce == nonces[msg.sender]++, InvalidNonce()); // consumes nonce, no event
    // ...
}
```

The security-relevant fact — which witness signature was redeemed at which `(sender, nonce, asset, to, amount)` tuple — is unobservable on-chain. The inner `CollateralToken.Wrapped/Unwrapped` events carry `caller = ramp` (not the end user), no nonce, and no signer, so they cannot reconstruct the authorization.

Emit a ramp-level event on `wrap()` / `unwrap()` including the consumed nonce, recipient and other relevant variables.



Client Commentary

Adding ramp-level events would improve auditability of witness-authorized nonce consumption, but we currently use the ramp and do not need this.

[L-28] `ResultReported.resolver` is inconsistent for contract-derived resolutions

The `BaseModuleEvents.ResultReported` event documents its indexed `resolver` field as "The resolver that reported". Because it is indexed, off-chain consumers filter on it. The event is emitted from five sites across three contracts, which disagree on what to put in `resolver` for resolutions the contract derives rather than a resolver actually reporting:

```
// BinaryModule.sol:reportResult() – a real resolver reported
emit ResultReported(msg.sender, _conditionId, _result);           // resolver = actual
caller

// NegRiskModule.sol:resolveConditionToNo() – permissionless force-to-NO, derived by the
contract
emit ResultReported(address(this), _conditionId, result_);       // resolver = the
module

// NegRiskModule.sol:_finalizeNegriskResolution() – a real resolver/bridge reported
emit ResultReported(msg.sender, _conditionId, _result);           // resolver = actual
caller

// NegRiskModule.sol:_finalizeNegriskResolution() – synthetic "Other" condition, auto-derived
by the contract
emit ResultReported(msg.sender, fallbackConditionId, fallbackResult); // resolver =
msg.sender (inaccurate)

// BaseMigrationMixin.sol:_finalizeMigrationResolution() – derived from legacy CTF payouts by
the contract
emit ResultReported(address(this), _conditionId, _result);       // resolver = the
module
```

Across these sites, contract-derived resolutions are attributed inconsistently. Two of them — `resolveConditionToNo()` and the migration finalizer — use `address(this)` as a sentinel meaning "no resolver reported; the contract derived this". But the synthetic `Other` condition is also computed entirely by the contract — nobody "reported" it — yet it is attributed to whoever happened to trigger the final real resolution (`msg.sender`).

An indexer filtering `ResultReported` by `resolver` will mis-attribute the synthetic-Other resolution to an arbitrary resolver/bridge address, and cannot consistently distinguish contract-derived resolutions from resolver-reported ones.

A second, separate problem compounds this: there are two distinct events both named `ResultReported` in the codebase, with different signatures and unrelated semantics:



```
// src/modules/abstract/BaseModule.sol – emitted when a module records a final payout vector
event ResultReported(address indexed resolver, ConditionId indexed conditionId, uint256[]
result);

// src/oracle/abstract/OracleAggregatorEvents.sol – emitted when a reporter module submits a
single vote
event ResultReported(bytes32 indexed requestId, address indexed module, bytes32 resultHash,
uint256 votes);
```

Because the parameter lists differ, the two have different topic0 hashes and are technically distinguishable on-chain. But reusing the exact name `ResultReported` for two unrelated concepts is a foot-gun for ABI tooling, indexers, and anyone reading logs by name.

`resolveMigrationCondition()` is an additionally affected path: this function is permissionless once the legacy CTF payout exists, so any arbitrary keeper or user address can call it. Because `_finalizeNegriskResolution()` emits `ResultReported(msg.sender, ...)` for migration-derived resolutions, an arbitrary external address is recorded in the event log as the reporter for migrated conditions, rather than `address(this)` as the sentinel indicating a contract-computed result.

Use `address(this)` when emitting `ResultReported` for the synthetic "Other" condition in `NegRiskModule.sol:_finalizeNegriskResolution()`. Additionally, rename one of the two `ResultReported` events so the two concepts are unambiguous by name.

[L-29] ERC-1271 signature validation forwards all available gas

`_isValidERC1271()` performs the `isValidSignature` call with `gas()`, forwarding the entire remaining gas of the matching transaction to the signer contract:

```
// src/exchange/Exchange.sol
function _isValidERC1271(address signer, bytes32 hash, bytes calldata sig) internal view
returns (bool isValid) {
    assembly ("memory-safe") {
        let m := mload(0x40)
        // `isValidSignature(bytes32,bytes)` selector in the top 4 bytes.
        let f := shl(224, 0x1626ba7e)
        mstore(m, f)
        mstore(add(m, 0x04), hash)
        let d := add(m, 0x24)
        mstore(d, 0x40) // Offset of `signature` in the ABI-encoded call.
        mstore(add(m, 0x44), sig.length)
        calldatacopy(add(m, 0x64), sig.offset, sig.length)
    @>
        isValid := staticcall(gas(), signer, m, add(sig.length, 0x64), d, 0x20)
        isValid := and(eq(mload(d), f), isValid)
    }
}
```

(Because of EIP-150's 63/64 rule the signer actually receives all but ~1/64 of the remaining gas, which for any realistic match is effectively the entire budget.)



The function already hardens this `staticcall` against a malicious signer in one dimension — it caps the copied returndata at 32 bytes (`d, 0x20`), which avoids copying an oversized return payload into the memory. The gas dimension is left open: the signer is a `staticcall` target that may perform gas-heavy computation, so a `POLY_1271` order whose `maker` is a contract that burns gas and then returns the magic value `0x1626ba7e` passes validation while the operator's transaction bears the burned gas.

The griefing manifests in one of two ways, depending on how much gas the signer burns relative to the block gas limit and the operator's chosen transaction gas limit:

- If the burn makes the whole match exceed the block gas limit, no builder can include the transaction — the operator simply cannot land that match. Every honest maker and the taker co-batched with the malicious order are denied service until the operator drops the offending order and re-batches.
- If the burn stays under the block limit, the transaction can be included in a block and the operator pays for it. The operator's exposure is capped by the transaction gas limit they set: if the burn pushes consumption past that limit the call reverts out-of-gas, and if it stays under, the match succeeds at an inflated cost.

Impact is bounded and operator-gated. Order selection is entirely the operator's: an attacker cannot insert its malicious order into a third party's match — only post a maker order attractive enough for the operator to pick.

Recommendation: Forward a bounded gas stipend to the ERC-1271 `staticcall` instead of all remaining gas, so a single signer cannot consume the transaction's entire gas budget.

Client Commentary

Supported `POLY_1271` usage is limited operationally to Polymarket Deposit Wallets; arbitrary ERC-1271 signer contracts are not part of the supported order flow (we perform offchain checks).

[L-30] Compress Function Silently Destroys Dust Positions in YES Branch

Description

The `CombinatorialModule::compress` function in the YES branch silently destroys position tokens due to rounding truncation, with no warning or revert to alert the user.

The Issue:

When compressing a YES combinatorial position after partial resolutions:

```
positionAmount = FixedPointMathLib.mulDiv(_amount, payoutFactor, PAYOUT_FACTOR_DENOMINATOR);
if (positionAmount != 0) {
    if (remainingCount == 0) {
        collateralOut = positionAmount;
    } else {
```



```
        newPositionId = _storeLegsFromMemory(trimmed).computePositionId(0);
    }
}
// Position burned regardless:
POSITION_MANAGER.burn(_positionId, _amount);
```

When `_amount * payoutFactor < PAYOUT_FACTOR_DENOMINATOR (1e36)`, `mulDiv` rounds down to 0. The function then:

1. Sets `positionAmount = 0`
2. Mints **zero** collateral and **no** new position
3. **Still burns** the original position

The caller loses their entire position token with zero output and no revert.

Critical Distinction:

This is **not** the legitimate case where `payoutFactor == 0` (meaning YES is genuinely worth zero after all legs resolve to zero). Here, `payoutFactor` is legitimately non-zero (e.g., 0.5e36 or higher), but rounding artifacts on small amounts truncate to zero.

Example Scenario: - User compresses 1 wei of a position with `payoutFactor = 0.5e36` -

`mulDiv(1, 0.5e36, 1e36) = 0` (rounding down) - Position burned, 0 collateral minted, 1 wei lost silently

Recommendations

Option 1: Block dust positions entirely

Revert if the compressed output would be zero while the payout factor is non-zero:

```
require(positionAmount > 0 || payoutFactor == 0, "CompressedToDust");
```

This forces users to use `redeem` for precise, full-amount payouts when dust risk exists.

Option 2: Skip burn on zero output

Alternative: burn only if output is minted:

```
// Mint the compressed outputs before pulling and burning the old position.
if (collateralOut != 0) COLLATERAL_TOKEN.mint(_to, collateralOut);
if (PositionId.unwrap(newPositionId) != 0) POSITION_MANAGER.mint(_to, newPositionId,
positionAmount);

// Callback to pull old position
if (_callbackReceiver != address(0)) {
    ICombinatorialModuleCallback(_callbackReceiver).compressCallback(msg.sender, _amount,
_positionId, _data);
}
```



```
// Only burn if we produced output  
require(collateralOut > 0 || PositionId.unwrap(newPositionId) != 0, "CompressedToDust");  
POSITION_MANAGER.burn(_positionId, _amount);
```

Option 1 is preferred because it makes the constraint explicit upfront and clearly communicates that `redeem` should be used for precise payouts.

Client Commentary

Acknowledge. Maximum loss is less than one base unit per compression call; repeated one-leg compressions are bounded by MAX_LEGS (50) for one canonical position lifecycle.